A 4–5 minute spoken walkthrough with timings, the tab-by-tab map, and a rapid-fire Q&A sheet. Speak the quoted blocks; keep the tables as your glance notes.

1 · Opening 30 sec

> "My project is **CampusOS** — a College Management System covering students, faculty, departments, courses, offerings, enrollments and grades, plus attendance and a library module. What I'll emphasize today is the part I personally built: **the frontend, the backend integration layer, and codebase management**. The distinctive thing is that it is not one demo — it is **one backend contract served by two independent frontends**, which forced me to keep integration discipline."

2 · Tech stack 60 sec

> "There are two builds in my repo. **The build I'm demonstrating** is Next.js 15 with React 19 on Node 26. The database is **SQLite running inside the Node process itself** using the official `node:sqlite` module — no separate database server; API and DB share one runtime. Data lives in a WAL-mode SQLite file, and schema, seed and migrations are SQL files under version control. Styling is **Tailwind CSS** with design tokens — charcoal surfaces, one flat green accent, no gradients: a clean, tool-like UI. **The second build** is a classic split stack: React + Vite frontend, FastAPI with SQLAlchemy and Pydantic, and **PostgreSQL 16 in Docker**. Both builds speak almost the same REST contract — that's my proof the integration layer is clean. Everything else is small: axios for HTTP, recharts for charts, lucide-react for icons, react-hook-form for forms. Testing is Node's built-in test runner plus Puppeteer browser smoke tests."

Component	Main build (demoed)	Second build
Frontend	Next.js 15 App Router, React 19, Tailwind 3	React + Vite, Tailwind, Recharts
Backend	Next.js API routes (Node 26)	FastAPI, SQLAlchemy, Pydantic
Database	SQLite via `node:sqlite`, WAL mode	PostgreSQL 16 (Docker)
HTTP client	axios (one shared instance)	axios
Tests	14 Node suites + Puppeteer smoke (24 checks)	83 pytest suites

3 · How the frontend is made 90 sec

> "The frontend is a single admin application rendered inside one smart root component, `CollegeApp.jsx` — a state machine with three states: *loading* (calls `/auth/me` to hydrate the session),

> *unauthenticated* (Login screen), *authenticated* (student portal or admin workspace based on the server-side role). So authorization isn't cosmetic — the React tree itself is different per role.

> "The key architectural idea is the **generic ResourcePage component**: fourteen reusable building blocks — DataTable, FormModal, ConfirmDialog, Pagination, Badge, Field, EmptyState, ErrorState, Skeleton — plus one config file, `resources.js`, declaring every entity as data: endpoint, columns, form fields, validation, sort, delete warning. A Student page and a Course page are the same component fed different config. I added attendance and the library module with roughly 15 lines of config and zero new components.

> "Data flow is one-directional. All HTTP goes through one axios instance in `services/api.js`. Failures are normalized there: a 401 fires a global session-expired event; 409/422 become readable field messages; and **only GET requests auto-retry** — mutations are never silently re-sent, because a retried POST could create a duplicate record."

- **Routing:** one catch-all page resolves `/students`, `/students/41`, `/reports`, `/settings` into a shared shell — fixed sidebar, topbar with theme toggle, shared toast notifications.
- **Detail screens:** students, faculty, courses, semesters, offerings each have a detail view with related records (enrollments, offerings, grades).

4 · What each tab does 60 sec

Tab	What it does
Dashboard	DB aggregates: student/faculty/course counts, enrollment trends (Recharts).
Students · Faculty · Departments	Full CRUD, search, sort, pagination; students & faculty FK-linked to departments.
Courses → Offerings → Enrollments → Grades	The academic chain: a course offered per semester with section & capacity; students enroll into an offering; grades attach to enrollments. **Capacity is enforced in the DB with triggers + immediate transactions** — a lost race gets a readable 409.
Semesters	Academic calendar anchor for offerings.
Attendance	Per-enrollment date records (Present / Absent / Late / Excused).
Library books / loans	Catalog plus issued-book tracking with due dates and overdue flags.
Reports	SQL aggregation: per-student GPA, grade distribution, semester counts; CSV export.
Student access	Provision a student's login, linked to their student record.
SQL activity	Live audit of the last 200 SQL statements executed, excluding password/session tables.
Settings	Environment info + one-click download of the whole database as SQL (reproducible demo state).

> "The **student role** sees none of that — a four-tab read-only portal: Overview, My courses with grades, Attendance with a computed rate, Library. Every query is filtered server-side by the student ID bound to the account, so account A physically cannot read account B's rows."

5 · Auth + closing 45 sec

> "Authentication is real, not demo-mocked: **scrypt-hashed passwords, server-side sessions in the database, HttpOnly SameSite cookies,** cross-origin protection, and login throttling per account. Plain-text passwords exist nowhere. Engineering discipline wraps it up: **14 Node test suites** covering CRUD, constraints, capacity races, authorization and the audit trail; a **24-check Puppeteer smoke test** in real Chromium; feature-scoped git commits; and personal data — the live SQLite file and SQL snapshots — is deliberately gitignored. So: one contract, two stacks, a config-driven frontend, database-enforced integrity, and tests that prove both."

Honesty line (use if pressed): "This is a functioning local administrative demo — real auth, real constraints, real tests. For production I'd add HTTPS termination, proxy rate limiting, and a multi-instance session store."

6 · Rapid-fire Q&A sheet

"Why SQLite in one build and PostgreSQL in the other?"

The Next build is a self-contained single-runtime deployment — SQLite via `node:sqlite` needs no server, ideal for a portable demo. The formal build uses PostgreSQL for real server features: concurrent transactions, `SELECT … FOR UPDATE` row locks, views, richer SQL. Same logical schema, both SQL, both with enforced foreign keys and triggers.

"Where is the backend? It looks like one repo."

Next.js API routes under `app/api/[...path]/route.js` *are* the backend — a server-side handler that opens the DB, runs a dispatch table, returns JSON. Server code, just shipped inside the Next app instead of a separate process like FastAPI in build two.

"Show me the integration layer."

Three files: `services/api.js` (single axios instance, error normalization, retry policy) · `lib/api.mjs` (server dispatch, validation, friendly constraint errors) · `resources.js` (the contract between them).

"How do you prevent SQL injection?"

Every query is parameterized with `?` placeholders — user input is never concatenated into SQL. Only whitelisted column names from internal config reach identifier positions. Tests assert malicious input fails safely.

"What if the DB goes down mid-session?"

503 → UI shows "Cannot reach the server, your records are safe" with a retry button. GETs auto-retry once with backoff; mutations show an error and stay on the form — nothing re-submitted, nothing lost.

"Why no Person → Student/Faculty inheritance?"

Specialization was deliberately **not** modeled: student and faculty have different required fields and lifecycles, so they are separate tables with their own FKs to department. A defensible normalization decision, not an oversight.

"Why two ER diagrams?"

Conceptual (Chen notation: entities, relationships, attributes) vs relational (actual tables, PKs, FKs, cardinality). Styled after my reference report; domain tables and the four auth tables are covered separately.

"What are you proudest of?"

Pick one: capacity race prevention at the DB level (two simultaneous last-seat enrollments — one succeeds, one gets a readable 409), or the config-driven architecture — "I added three entities and ten screens without writing a new component."

"Concurrency — explain."

SQLite build: `BEGIN IMMEDIATE` transactions plus capacity triggers. PostgreSQL build: `SELECT … FOR UPDATE` row locks plus a trigger. Both guarantee the last seat can never be double-booked.

"How do you know it works?"

14 Node suites (CRUD, constraints, capacity races, authorization, audit trail) = 14/14 pass; 24-check Puppeteer smoke against port 3000; live SQL snapshot restores into a fresh SQLite file with a clean foreign-key check.

7 · Numbers to have ready

Fact	Value
Domain tables	12 (9 academic + attendance, library_book, library_loan)
Auth/support tables	4 (portal_account, portal_session, login_throttle, portal_migration)
Password/token storage	scrypt hashes + SHA-256 token hashes only
Session lifetime	8 hours, HttpOnly · SameSite=Strict
Login throttle	5 failed attempts → 15-minute lockout (hashed identifier only)
SQL audit viewer	last 200 statements, auth tables excluded
DB snapshot	`GET /api/database/export.sql`, auto-refreshed on every mutation
Test status at push	14/14 Node suites · Next build ✓ · Vite build ✓ · UI smoke 24/24
Repo	github.com/Bhanu4417/DBMS-PROJCT (main)